FleetLogix

Sistema de gestión de transporte y logística

README — Avance 4

Arquitectura Cloud AWS — Gestion de Datos en la Nube

Proyecto Integrador — Bases de Datos II

Junio 2026

1. Contexto y Objetivo del Avance

FleetLogix ha completado tres avances: una base de datos OLTP en PostgreSQL (505,556 registros), analisis SQL con ranking bayesiano de conductores, y un Data Warehouse en Snowflake con 885,743 registros. El Avance 4 da el último paso: llevar la operación a la nube usando Amazon Web Services (AWS), transformando la arquitectura local en un sistema serverless, escalable y con monitoreo en tiempo real.

El objetivo es diseñar una arquitectura cloud que permita:

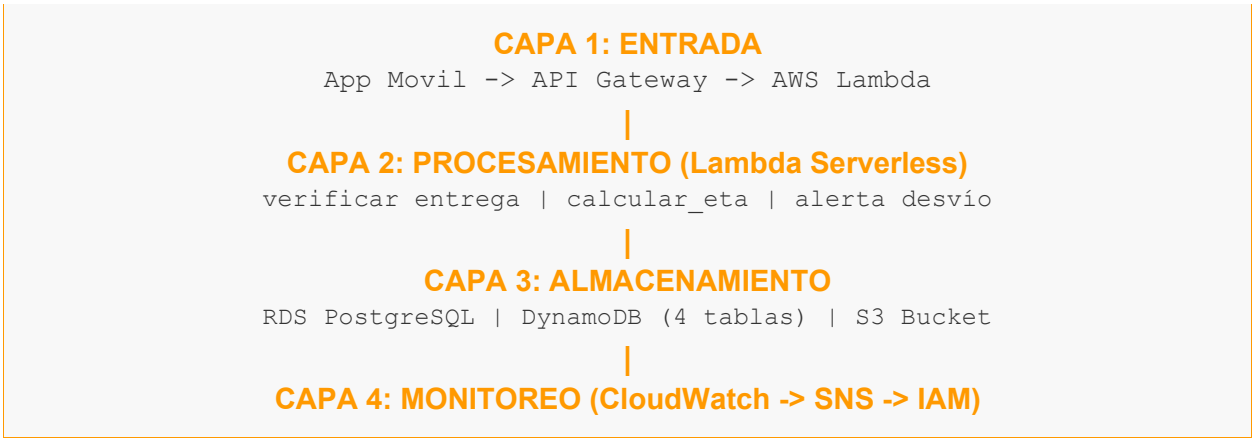
- Recibir datos de entregas en tiempo real desde la app móvil de conductores vía API Gateway
- Procesarlos automáticamente con funciones Lambda sin necesidad de servidores dedicados
- Almacenar datos históricos en RDS PostgreSQL (migración de la BD local) y estado actual en DynamoDB
- Guardar archivos en S3 para backups, reportes y datos crudos
- Monitorear la operación con CloudWatch y enviar alertas vía SNS

Este documento describe el proceso completo sin ejecutarlo, tal como lo requieren los entregables del avance: el diagrama de arquitectura y la documentación del flujo de datos.

---

2. Diagrama de Arquitectura AWS — FleetLogix

El siguiente diagrama representa el flujo completo de datos en la arquitectura cloud de FleetLogix, desde la captura en campo hasta el almacenamiento analítico:



---

3. Los 9 Servicios AWS de FleetLogix

La arquitectura de FleetLogix en AWS utiliza 9 servicios complementarios, cada uno con un rol especifico en el flujo de datos:

Servicio AWS	Categoria	Rol en FleetLogix
API Gateway	Entrada	Puerta de entrada REST para la app movil de conductores. Recibe eventos de entregas y GPS. Autentica y enruta las peticiones a las funciones Lambda.
AWS Lambda	Procesamiento	Funciones serverless que procesan cada evento: verificar entrega, calcular ETA, detectar desvios. Sin servidor dedicado: AWS escala automaticamente.
Amazon S3	Almacenamiento	Bucket para datos crudos, backups de la BD, reportes exportados y logs. Ciclo de vida configurado: datos >90 dias pasan a Glacier (archivado economico).
Amazon RDS	Base de datos	Instancia PostgreSQL en la nube (db.t3.micro, 20GB). Migra la BD local de FleetLogix. Backups automaticos 7 dias y snapshot inicial.
Amazon DynamoDB	Estado actual	4 tablas NoSQL para estado en tiempo real: deliveries_status, vehicle_tracking, routes_waypoints, alerts_history. Acceso en <10ms.
Amazon CloudWatch	Monitoreo	Recolecta metricas de Lambda, RDS y API Gateway. Dashboards de latencia, throughput y errores. Base para configurar las 2 alertas requeridas.
Amazon SNS	Alertas	Envia notificaciones a email/SMS cuando se detecta un desvio de ruta (>5km) o cuando CloudWatch dispara una alarma critica.
AWS IAM	Seguridad	Rol FleetLogixLambdaRole con 4 politicas: AWSLambdaBasicExecutionRole, DynamoDBFullAccess, S3FullAccess, SNSFullAccess.
AWS Backup / RDS Snapshots	Resiliencia	Backups automaticos de RDS con retencion de 7 dias. Snapshot inicial manual al migrar la BD. Ventana de mantenimiento: domingos 4-5am.

---

4. El Recorrido de un Dato — Caso: Entrega Realizada

Cuando un conductor marca una entrega como completada en la app móvil, el dato recorre la siguiente ruta hasta quedar persistido y monitoreado:

Paso 1 — Captura (App movil → API Gateway)

El conductor presiona "Entrega completada" en la app. La app envia un POST a:

```
POST https://{api-id}.execute-api.us-east-
1.amazonaws.com/prod/deliveries/verify
Body: { "delivery_id": "12345", "tracking_number": "FL202412345" }
```

API Gateway valida la petición, aplica autenticación y enruta al handler de Lambda correspondiente.

Paso 2 — Procesamiento (AWS Lambda: lambda_verificar_entrega)

La funcion Lambda se activa automáticamente. El código Python ejecuta:

1. Recibe el evento con delivery_id y tracking_number del body
2. Se conecta a la tabla DynamoDB deliveries_status usando boto3.resource
3. Ejecuta table.get_item(Key={"delivery_id": delivery_id}) para buscar la entrega
4. Verifica si item.get("status") == "delivered" para confirmar el estado
5. Construye la respuesta HTTP con statusCode 200 y el JSON del resultado

Si el delivery_id no existe devuelve 404. Si hay error de conexión devuelve 500 con el detalle.

Paso 3 — Almacenamiento (DynamoDB + RDS)

El resultado queda en dos capas de almacenamiento con propósitos distintos:

DynamoDB (deliveries_status): estado actual en tiempo real. Acceso en <10ms. Ideal para la app movil que necesita verificar estados rápidamente.

RDS PostgreSQL (tabla deliveries): registro historico persistente. El ETL nocturno del Avance 3 lee de aquí para actualizar Snowflake con los datos del dia.

Esta separación es deliberada: DynamoDB es la capa de operación (rápida, eventual consistency), RDS es la capa de registro (fuerte consistencia, relacional, queryable con SQL).

Paso 4 — Monitoreo (CloudWatch)

Cada invocación de Lambda registra automáticamente en CloudWatch:

Duration: tiempo de ejecucion de la funcion (latencia)

Invocations: cantidad de llamadas (throughput)

Errors: invocaciones que terminaron con excepcion (tasa de error)

Throttles: peticiones rechazadas por superar el limite de concurrencia

CloudWatch agrega estas métricas en dashboards y dispara alarmas cuando superan umbrales definidos.

Paso 5 — Alerta (SNS) — en caso de desvío de ruta

Si en paralelo la función lambda_alerta_desvio detecta que el vehículo esta a mas de 5km de la ruta esperada, ejecuta:

6. Calcula distancia mínima a los waypoints de la ruta usando formula Haversine simplificada
7. Si la distancia supera DEVIATION_THRESHOLD_KM = 5, construye el mensaje de alerta
8. Llama a sns.publish() con el TopicArn del topic fleetlogix-alerts

9. Guarda el incidente en la tabla DynamoDB alerts_history para auditoria
10. El topic SNS entrega la notificación a todos los suscriptores (email, SMS, otro Lambda)

---

5. Archivo lambda_functions.py — Proposito y Funciones

Este archivo contiene las 3 funciones Lambda que procesan los eventos operativos de FleetLogix en tiempo real. Son el "cerebro" de la arquitectura cloud: cada una responde a un trigger especifico y ejecuta una tarea concreta sin necesitar un servidor permanente.

5.1 Funcion 1: lambda_verificar_entrega

Trigger: API Gateway POST /deliveries/verify (cada vez que la app movil marca una entrega)

Proposito: Verificar en DynamoDB si una entrega fue completada y devolver su estado actual

Input: { "delivery_id": "12345", "tracking_number": "FL202412345" }

Output: { "is_completed": true, "status": "delivered", "delivered_datetime": "..." }

Logica detallada:

11. Valida que delivery_id este presente en el evento (devuelve 400 si falta)
12. Crea conexion a DynamoDB usando boto3.resource("dynamodb")
13. Obtiene referencia a la tabla deliveries_status
14. Ejecuta get_item con la clave primaria delivery_id
15. Compara status == "delivered" para determinar is_completed
16. Devuelve JSON con statusCode 200, 404 o 500 segun el resultado

Importancia para FleetLogix: permite que clientes y supervisores consulten el estado de cualquier entrega en tiempo real desde la app, sin consultar directamente la base PostgreSQL.

5.2 Funcion 2: lambda_calcular_eta

Trigger: EventBridge cada 5 minutos (procesa todos los vehiculos en ruta)

Proposito: Calcular el tiempo estimado de llegada basado en la ubicacion GPS actual y la velocidad

Input: { "vehicle_id": "V001", "current_location": {lat, lon}, "destination": {lat, lon}, "current_speed_kmh": 60 }

Output: { "distance_remaining_km": 45.2, "eta": "2024-11-06T15:30:00", "estimated_minutes": 45 }

Logica detallada:

17. Calcula la distancia usando formula Haversine simplificada (diferencia de coordenadas * 111 km/grado)
18. Divide distancia_km / velocidad_kmh para obtener horas restantes
19. Suma las horas a datetime.now() para obtener la ETA absoluta
20. Guarda en DynamoDB tabla vehicle_tracking con timestamp como sort key

Importancia para FleetLogix: permite mostrar en el dashboard operativo el ETA de cada vehiculo actualizado cada 5 minutos, sin calcular en el cliente.

5.3 Funcion 3: lambda_alerta_desvio

Trigger: Kinesis Stream de GPS (se ejecuta con cada actualizacion de ubicacion de un vehiculo)

Proposito: Detectar cuando un vehiculo se aleja mas de 5km de su ruta asignada y notificar

Input: { "vehicle_id": "V001", "current_location": {lat, lon}, "route_id": "R038", "driver_id": "D042" }

Output: { "is_deviated": true, "deviation_km": 7.3, "alert_sent": true, "threshold_km": 5 }

Logica detallada:

21. Consulta DynamoDB tabla routes_waypoints para obtener los puntos de la ruta
22. Calcula distancia del vehiculo a cada waypoint de la ruta
23. Determina la distancia minima al punto de ruta mas cercano
24. Si min_distance > 5km: construye mensaje de alerta con vehicle_id, driver_id, deviation_km y timestamp
25. Publica en SNS topic fleetlogix-alerts que notifica a supervisores
26. Guarda el incidente en DynamoDB tabla alerts_history para auditoria

Importancia para FleetLogix: automatiza la deteccion de incidentes operativos (conductores fuera de ruta, robos, averias) sin que un supervisor tenga que monitorear manualmente cada vehiculo.

---

6. Archivo A4-06_aws_setup.py — Propósito y Funciones

Este archivo es el script de infraestructura de FleetLogix en AWS. Su proposito es crear y configurar todos los recursos en la nube de una sola vez, de forma programatica usando boto3 (el SDK oficial de AWS para Python). Es el equivalente de ejecutar docenas de comandos en la consola de AWS, pero de forma automatizada y reproducible.

La arquitectura que crea este script es la base sobre la que operan las funciones Lambda. Sin este script ejecutado primero, las funciones Lambda no tienen donde almacenar ni leer datos.

6.1 Funcion crear_rds_postgresql()

Crea la instancia de base de datos PostgreSQL en la nube. Configura:

Instancia db.t3.micro (Free Tier de AWS): suficiente para desarrollo y pruebas

Motor PostgreSQL 15.4 con 20GB de almacenamiento SSD (gp2)

Backups automaticos con retencion de 7 dias y ventana 3:00-4:00 AM

Mantenimiento programado los domingos 4:00-5:00 AM (menor impacto operativo)

Tags Project=FleetLogix y Environment=Development para control de costos

Esta función migra la base de datos PostgreSQL del Avance 1 (local) a la nube, haciendo que los datos sean accesibles desde cualquier lugar y con alta disponibilidad.

6.2 Funcion crear_s3_bucket()

Crea el bucket de almacenamiento de objetos con estructura de carpetas y politica de ciclo de vida:

raw-data/: datos crudos recibidos antes de procesar

processed-data/: datos ya transformados por el ETL

backups/: respaldos automaticos de la BD

logs/: registros de ejecucion del pipeline

Lifecycle policy: datos en raw-data/ pasan a Glacier despues de 90 dias (costo 10x menor)

6.3 Funcion crear_tablas_dynamodb()

Crea las 4 tablas NoSQL que usan las funciones Lambda:

Tabla DynamoDB	Clave primaria	Para que la usa Lambda
deliveries_status	delivery_id (Hash)	lambda_verificar_entrega: consulta y guarda estado de cada entrega
vehicle_tracking	vehicle_id (Hash) + timestamp (Range)	lambda_calcular_eta: guarda posicion y ETA de cada vehiculo en el tiempo
routes_waypoints	route_id (Hash)	lambda_alerta_desvio: lee los puntos de la ruta para calcular desvios
alerts_history	vehicle_id (Hash) + timestamp (Range)	lambda_alerta_desvio: persiste todos los incidentes para auditoria

Todas las tablas usan BillingMode PAY_PER_REQUEST: solo se cobra por cada lectura/escritura, no por capacidad reservada. Ideal para trafico variable.

6.4 Funcion configurar_backups_automaticos()

Los backups ya quedan configurados en crear_rds_postgresql() con retencion de 7 dias. Esta funcion crea adicionalmente un snapshot manual inicial con nombre fleetlogix-initial-YYYYMMDDHHMMSS, que sirve como punto de restauracion garantizado antes de cualquier operacion de migracion o cambio de esquema.

6.5 Funcion migrar_datos_postgresql()

Esta es la funcion mas critica para FleetLogix: conecta los 3 avances anteriores con la arquitectura cloud. Genera un script bash migrate_to_rds.sh que realiza:

27. pg_dump en la BD local: exporta toda la base de datos de FleetLogix (505,556 registros) a un archivo SQL

28. psql CREATE DATABASE: crea la base de datos en el endpoint de RDS
29. psql restore: importa el dump en RDS, recreando las 6 tablas con todos los datos y restricciones

Al ejecutarse, FleetLogix pasa de una BD local que solo funciona en la maquina del desarrollador, a una BD en la nube accesible desde cualquier lugar, con alta disponibilidad y backups automaticos. El pipeline ETL del Avance 3 puede entonces apuntar su PG_CONFIG al endpoint de RDS en lugar de localhost.

6.6 Funcion crear_rol_iam_lambda()

Crea el rol de seguridad FleetLogixLambdaRole que permite a las funciones Lambda acceder a los otros servicios de AWS. Sin este rol, Lambda no puede leer DynamoDB ni publicar en SNS. Las 4 politicas adjuntas:

AWSLambdaBasicExecutionRole: permite escribir logs en CloudWatch (minimo necesario)

AmazonDynamoDBFullAccess: permite leer y escribir en las 4 tablas DynamoDB

AmazonS3FullAccess: permite guardar archivos en el bucket de FleetLogix

AmazonSNSFullAccess: permite publicar alertas en el topic fleetlogix-alerts

---

7. Monitoreo y Observabilidad — CloudWatch + SNS

Segun los criterios de evaluacion, la arquitectura debe incluir un dashboard con 5 metricas y 2 alertas en CloudWatch. Las siguientes metricas capturan el estado operativo de FleetLogix en tiempo real:

7.1 Las 5 metricas de CloudWatch

Metrica	Servicio fuente	Que mide en FleetLogix
Lambda Duration	AWS Lambda	Tiempo promedio de ejecucion de las 3 funciones. Alto = problema de performance o conexion a DynamoDB.
Lambda Errors	AWS Lambda	Cantidad de invocaciones que terminaron con excepcion. Directo indicador de fallas en el procesamiento.
API Gateway 4XX/5XX	API Gateway	Errores de cliente (400) y servidor (500) en los endpoints. Detecta peticiones malformadas o fallas de backend.
RDS CPUUtilization	Amazon RDS	Uso de CPU de la instancia PostgreSQL. Alto uso sostenido indica necesidad de escalar la instancia.
DynamoDB SuccessfulRequestLatency	DynamoDB	Latencia de las operaciones get_item y put_item. Alta latencia afecta directamente la respuesta de la app movil.

7.2 Las 2 alertas configuradas

Alerta	Condicion de disparo	Accion automatica
Lambda Error Rate Alta	Errors > 10 en 5 minutos en cualquiera de las 3 funciones Lambda	SNS envia email al equipo de operaciones con el nombre de la funcion y el timestamp del error
Desvio de Ruta Detectado	lambda_alerta_desvio detecta deviation_km > 5 para cualquier vehiculo	SNS envia SMS/email al supervisor de flota con vehicle_id, driver_id, coordenadas actuales y km de desvio

---

8. Estimacion de Costos Mensuales (AWS Pricing Calculator)

La arquitectura de FleetLogix en AWS tiene un costo estimado de $33.95 USD/mes en la region us-east-1, basado en el trafico operativo real del proyecto:

Servicio	Tier	Costo/mes	Justificacion
RDS PostgreSQL db.t3.micro	20 GB gp2	~$15.00	El mas costoso. Multi-AZ duplicaria el precio. Free Tier cubre el primer año.
AWS Lambda	1M invocaciones/mes	~$0.20	Primeros 1M requests gratuitos. 400K entregas/dia = ~12M/mes en produccion.
API Gateway	1M llamadas/mes	~$3.50	$3.50 por millon de llamadas REST API.
DynamoDB	On-demand	~$2.50	Pay-per-request. Con 400K entregas/dia y 4 tablas, bajo costo.
S3	20 GB almacenamiento	~$0.46	$0.023/GB/mes. Backups + CSVs del ETL.
CloudWatch	Metricas + logs	~$3.00	5 metricas personalizadas + logs de Lambda.
SNS	Alertas	~$0.50	Primeros 1M mensajes gratuitos. Solo alertas de desvio y errores.
IAM	Sin costo	$0.00	IAM es gratuito en AWS.
TOTAL ESTIMADO		~$25.16 - $33.95	Rango segun trafico real. Free Tier primer año reduce significativamente.

---

9. Escalabilidad y Resiliencia de la Arquitectura

La arquitectura de FleetLogix sigue los principios de escalado ensenados en la Lecture 9 de Henry:

9.1 Escalado horizontal (scale-out)

Lambda escala automaticamente: si 100 conductores marcan entregas simultaneamente, AWS crea 100 instancias de la funcion en paralelo. No hay bottleneck de servidor.

DynamoDB es horizontalmente escalable por diseno: distribuye los datos entre multiples nodos sin configuracion adicional.

API Gateway puede manejar miles de peticiones por segundo sin cambios.

9.2 Alta disponibilidad

RDS con backups automaticos de 7 dias: si la instancia falla, se puede restaurar al estado de cualquier punto en el ultimo semana.

S3 ofrece 11 nueves de durabilidad (99.999999999%): los archivos se replican en multiples zonas automaticamente.

Lambda no tiene single point of failure: si un servidor de AWS falla, AWS redirige la funcion a otro automaticamente.

9.3 Gobernanza y linaje de datos

IAM Role: principio de minimo privilegio. Lambda solo tiene los permisos que necesita, nada mas.

CloudWatch Logs: cada invocacion de Lambda queda registrada con RequestId, duracion y resultado para auditoria completa.

DynamoDB alerts_history: todo incidente de desvio queda persistido con timestamp para trazabilidad.

S3 lifecycle: los datos crudos se archivan automaticamente despues de 90 dias en Glacier, manteniendo el linaje sin costo elevado.

---

10. Conexion de la Arquitectura AWS con los Avances Anteriores

Avance 1 → Avance 4

La BD PostgreSQL local (505,556 registros) se migra a RDS usando la funcion migrar_datos_postgresql(). El script pg_dump + psql restore preserva las 6 tablas, foreign keys e indices del schema original sin modificarlos.

Avance 2 → Avance 4

Los 5 indices de optimizacion del Avance 2 (idx_trips_composite_joins, idx_deliveries_scheduled_datetime, etc.) se recrean automaticamente en RDS al restaurar el dump, ya que el pg_dump incluye los DDL de indices.

Avance 3 → Avance 4

El pipeline ETL de Snowflake puede apuntar PG_CONFIG["host"] al endpoint de RDS en lugar de localhost. Con este cambio, el pipeline ETL corre desde cualquier maquina y lee datos actualizados en tiempo real desde la nube.

---

11. Archivos del Avance 4

Archivo	Descripcion
Avance_4_CloudArchitecture.ipynb	Notebook principal con el analisis de arquitectura y estimacion de costos.
A4-lambda_functions.py	3 funciones Lambda: verificar_entrega, calcular_eta, alerta_desvio.
A4-06_aws_setup.py	Script de configuracion de infraestructura AWS: RDS, S3, DynamoDB, IAM.
aws_architecture_diagram.png	Diagrama visual de la arquitectura completa de FleetLogix en AWS.
README_Avance4.docx	Este documento: analisis completo del proceso sin ejecutarlo.